

Classification Model Evaluation

Confusion Matrix | Accuracy | Precision | Recall | AUC | ROC

	TRUE	FALSE
TRUE	True positive 	False positive (Type I error) 
FALSE	False negative (Type II error) 	True negative 

- TP: Algo predicts **Yes**, and the actual value is **Yes**!
- FP: Algo predicts **Yes**, but the actual value is **No**!
- FN: Algo predicts **No**, and the actual value is **Yes**!
- TN: Algo predicts **No**, and the actual value is **No**!

		Actual	
		CAT	NOT CAT
Predicted	CAT		
	NOT CAT		

- TP: Algo predicts **Yes**, and the actual value is **Yes**!
- FP: Algo predicts **Yes**, but the actual value is **No**!
- FN: Algo predicts **No**, and the actual value is **Yes**!
- TN: Algo predicts **No**, and the actual value is **No**!

A confusion matrix is a technique used in machine learning and statistical classification to evaluate the performance of a **classification model**. It is a specific table layout that allows visualization of the performance of an algorithm, typically used in supervised learning.

	Actual Positive	Actual Negative
Predicted Positive	TP	FP
Predicted Negative	FN	TN

- ❖ **True Positive (TP):** The number of correctly predicted positive samples.
 - ❑ Example: The model correctly predicts a positive instance as positive.
- ❖ **True Negative (TN):** The number of correctly predicted negative samples.
 - ❑ Example: The model correctly predicts a negative instance as negative.

A confusion matrix is a technique used in machine learning and statistical classification to evaluate the performance of a **classification model**. It is a specific table layout that allows visualization of the performance of an algorithm, typically used in supervised learning.

	Actual Positive	Actual Negative
Predicted Positive	TP	FP
Predicted Negative	FN	TN

- ❖ **False Positive (FP):** The number of samples incorrectly predicted as positive.
 - ❑ Example: The model predicts a negative instance as positive (Type I error).
- ❖ **False Negative (FN):** The number of samples incorrectly predicted as negative.
 - ❑ Example: The model predicts a positive instance as negative (Type II error).

The confusion matrix provides a comprehensive overview of how well your classification model is performing and helps in identifying the types of errors the model is making. By analysing the matrix, you can calculate several important metrics, including:

1. **Accuracy:** The proportion of the total number of predictions that were correct.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

2. **Precision (Positive Predictive Value):** The proportion of positive predictions that were actually correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

The confusion matrix provides a comprehensive overview of how well your classification model is performing and helps in identifying the types of errors the model is making. By analysing the matrix, you can calculate several important metrics, including:

3. **Recall (Sensitivity or True Positive Rate):** The proportion of actual positives that were correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN}$$

4. **F1 Score:** The harmonic mean of precision and recall.

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The confusion matrix provides a comprehensive overview of how well your classification model is performing and helps in identifying the types of errors the model is making. By analysing the matrix, you can calculate several important metrics, including:

5. **Specificity (True Negative Rate):** The proportion of actual negatives that were correctly identified.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

6. **False Positive Rate:** The proportion of actual negatives that were incorrectly identified as positives.

$$\text{False Positive Rate} = \frac{FP}{TN + FP}$$

In the context of a confusion matrix and classification tasks, the implications of false positives (FP) and false negatives (FN) can be significant, depending on the application.

False Positive (FP): A false positive occurs when **a model incorrectly predicts a positive outcome** for a negative instance. The dangers of false positives can be severe in some scenarios:

❑ Medical Diagnosis:

- ❖ Scenario: Predicting that a patient has a disease when they do not.
- ❖ Danger: This can lead to unnecessary stress, further invasive tests, and potentially harmful treatments for the patient, as well as unnecessary healthcare costs.

❑ Spam Detection:

- ❖ Scenario: Marking a legitimate email as spam.
- ❖ Danger: Important communications may be missed, leading to missed opportunities or misunderstandings.

In the context of a confusion matrix and classification tasks, the implications of false positives (FP) and false negatives (FN) can be significant, depending on the application.

False Negative (FN): A false negative occurs when a model incorrectly predicts a negative outcome for an instance that is positive. The dangers of false negatives can also be severe in various applications:

❑ **Medical Diagnosis:**

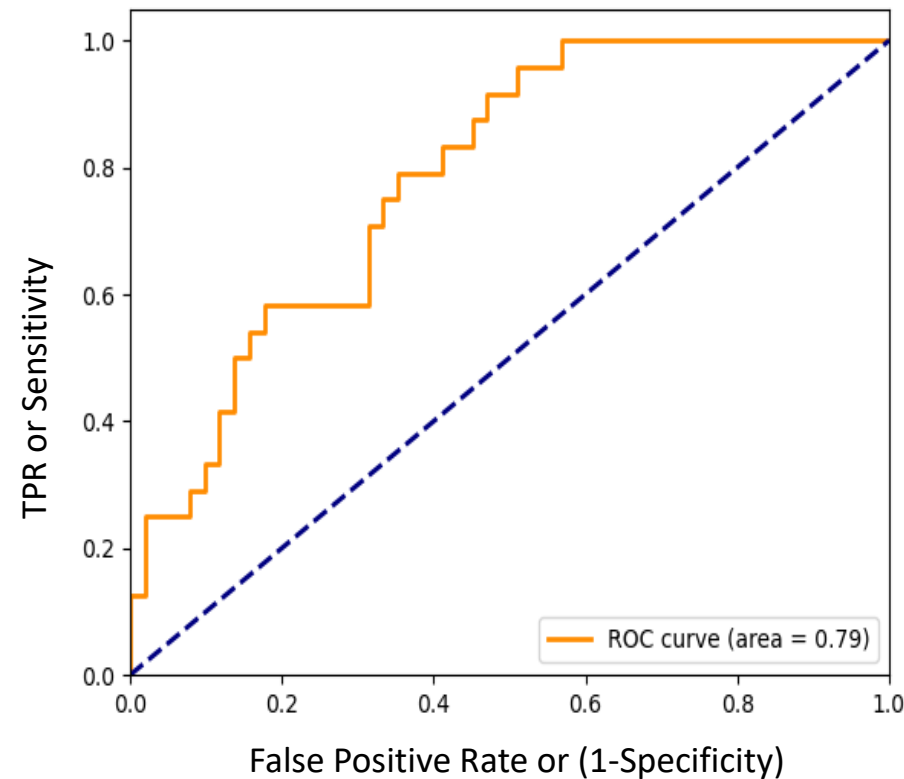
- ❖ Scenario: Predicting that a patient does not have a disease when they do.
- ❖ Danger: The disease may progress without treatment, leading to severe health consequences or death. Early detection and treatment opportunities are missed.

❑ **Security Systems:**

- ❖ Scenario: Failing to detect a malicious attack or malware.
- ❖ Danger: Actual threats go unnoticed, leading to potential breaches, data loss, or damage to systems and networks.

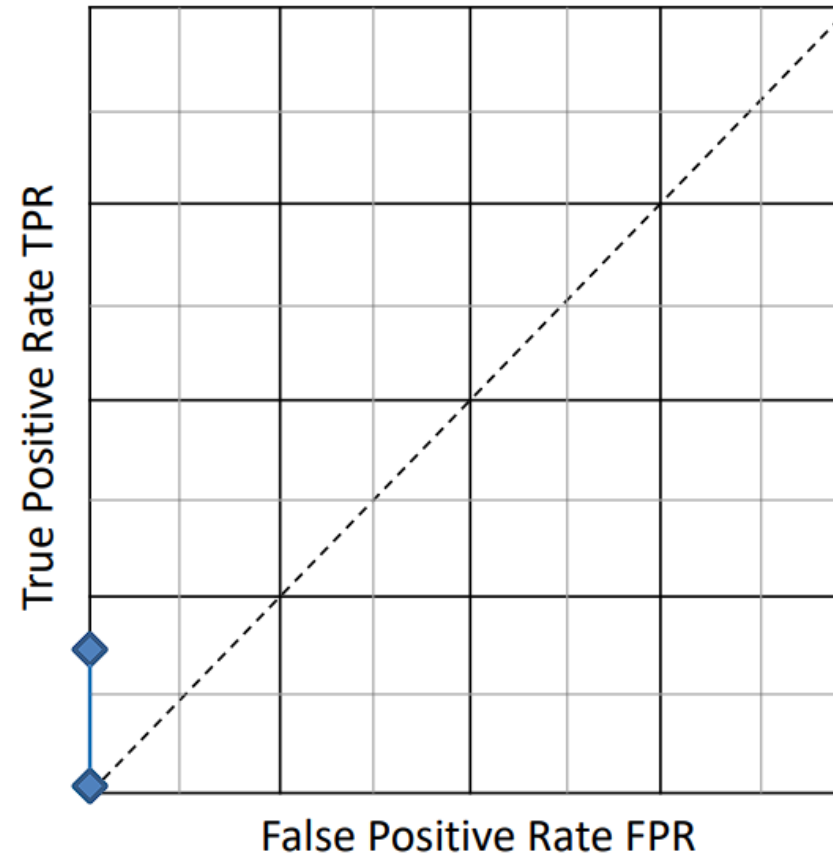
Receiver Operating Characteristic (ROC)

Receiver Operating Characteristic (ROC): Since **TPR is equivalent to Sensitivity** and **FPR is equal to $1 - \text{specificity}$** , the ROC graph is sometimes called the sensitivity vs ($1 - \text{specificity}$) plot.



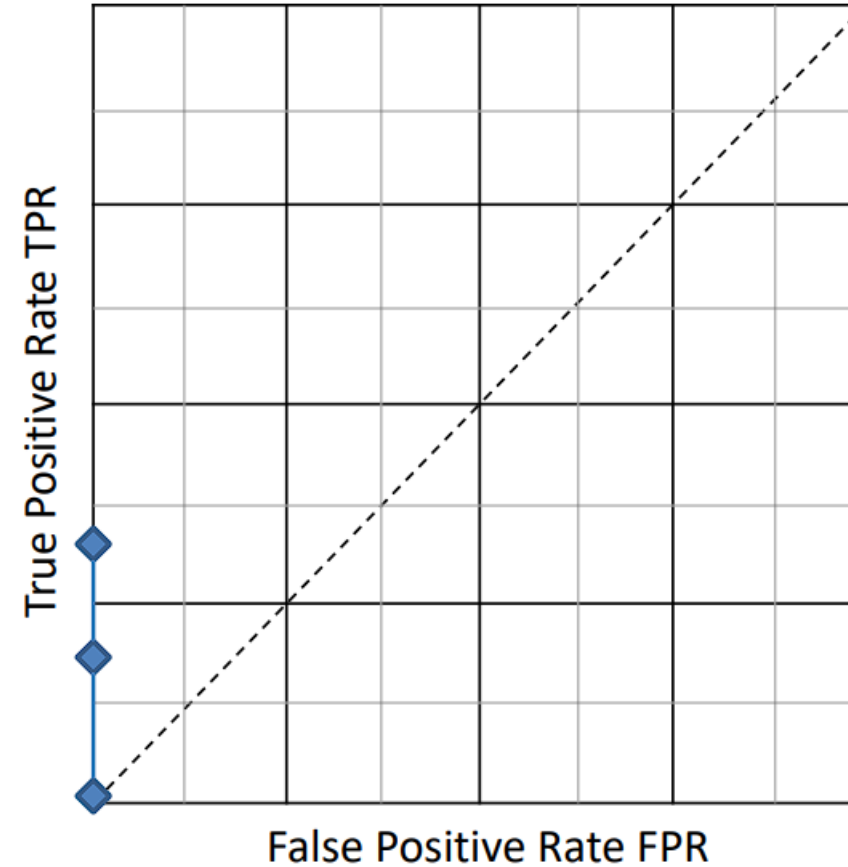
Receiver Operating Characteristic (ROC)

	Predict Correct	Score
1	Yes	0.95
2	Yes	0.86
3	Yes	0.69
4	No	0.65
5	Yes	0.59
6	No	0.52
7	No	0.39
8	No	0.28
9	Yes	0.15
10	No	0.06



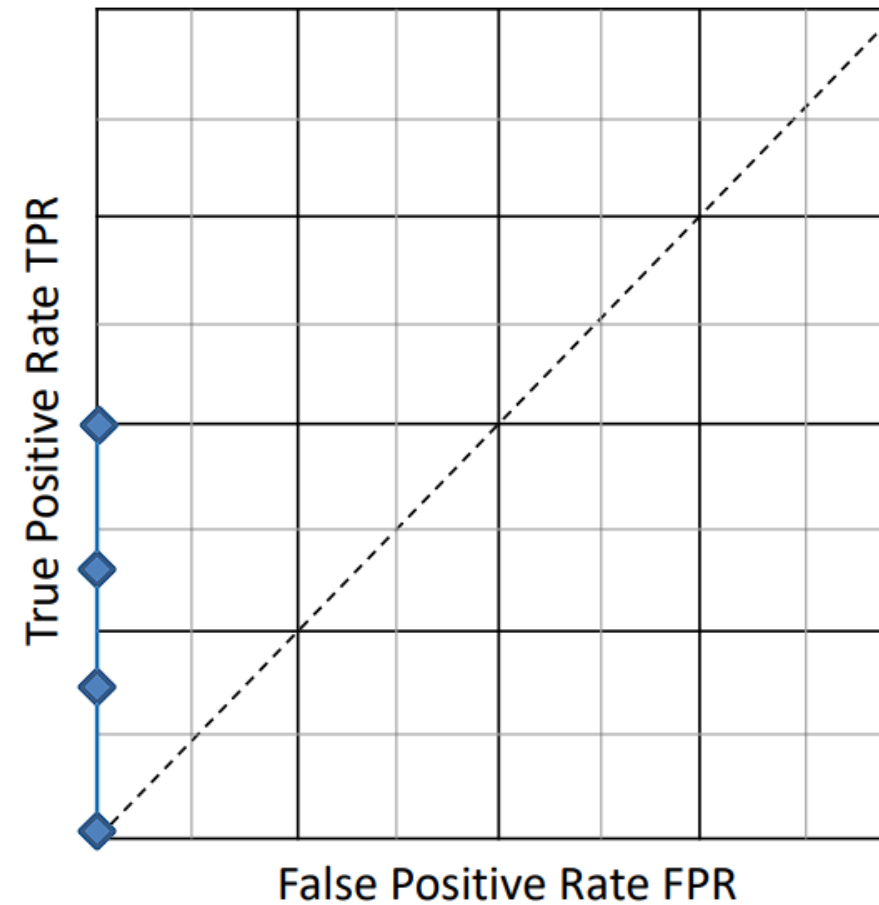
Receiver Operating Characteristic (ROC)

	Predict Correct	Score
1	Yes	0.95
2	Yes	0.86
3	Yes	0.69
4	No	0.65
5	Yes	0.59
6	No	0.52
7	No	0.39
8	No	0.28
9	Yes	0.15
10	No	0.06



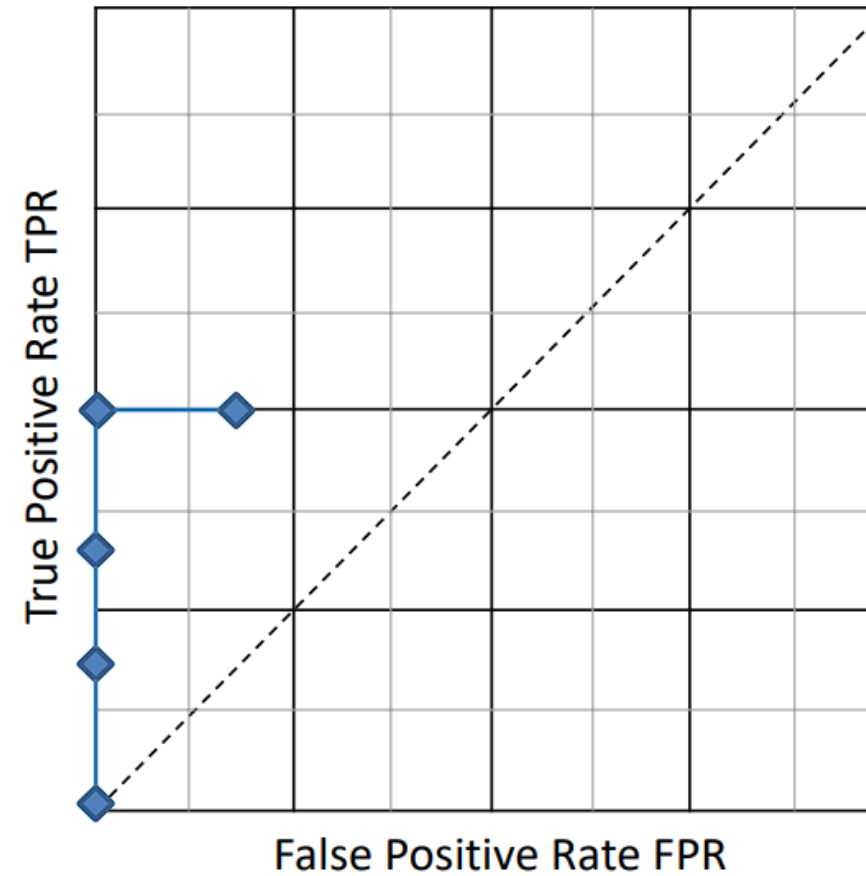
Receiver Operating Characteristic (ROC)

	Predict Correct	Score
1	Yes	0.95
2	Yes	0.86
3	Yes	0.69
4	No	0.65
5	Yes	0.59
6	No	0.52
7	No	0.39
8	No	0.28
9	Yes	0.15
10	No	0.06

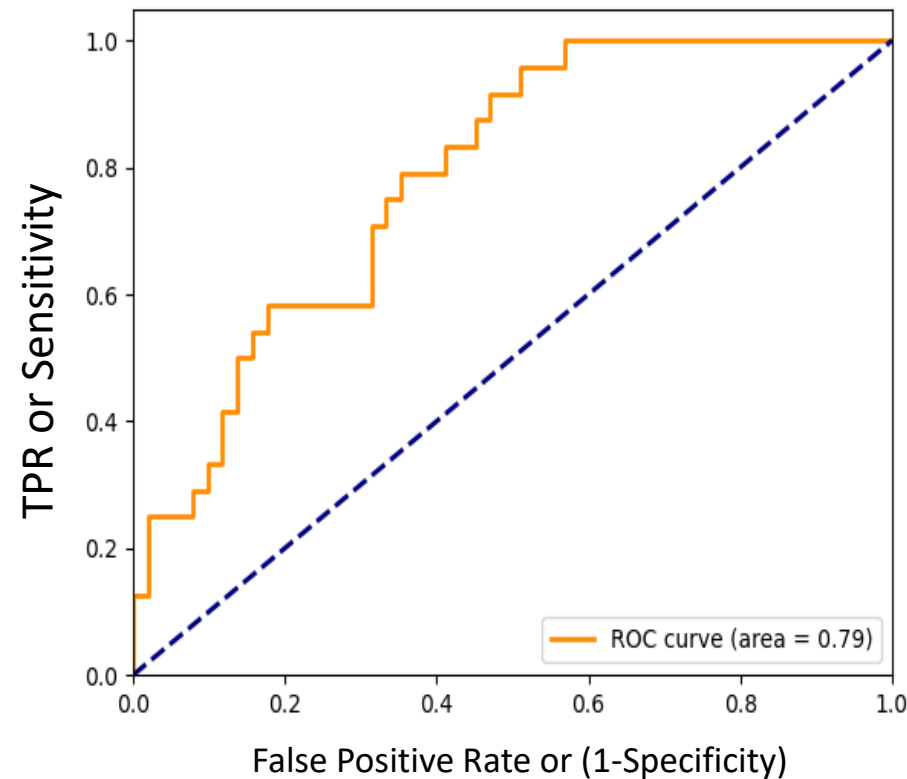


Receiver Operating Characteristic (ROC)

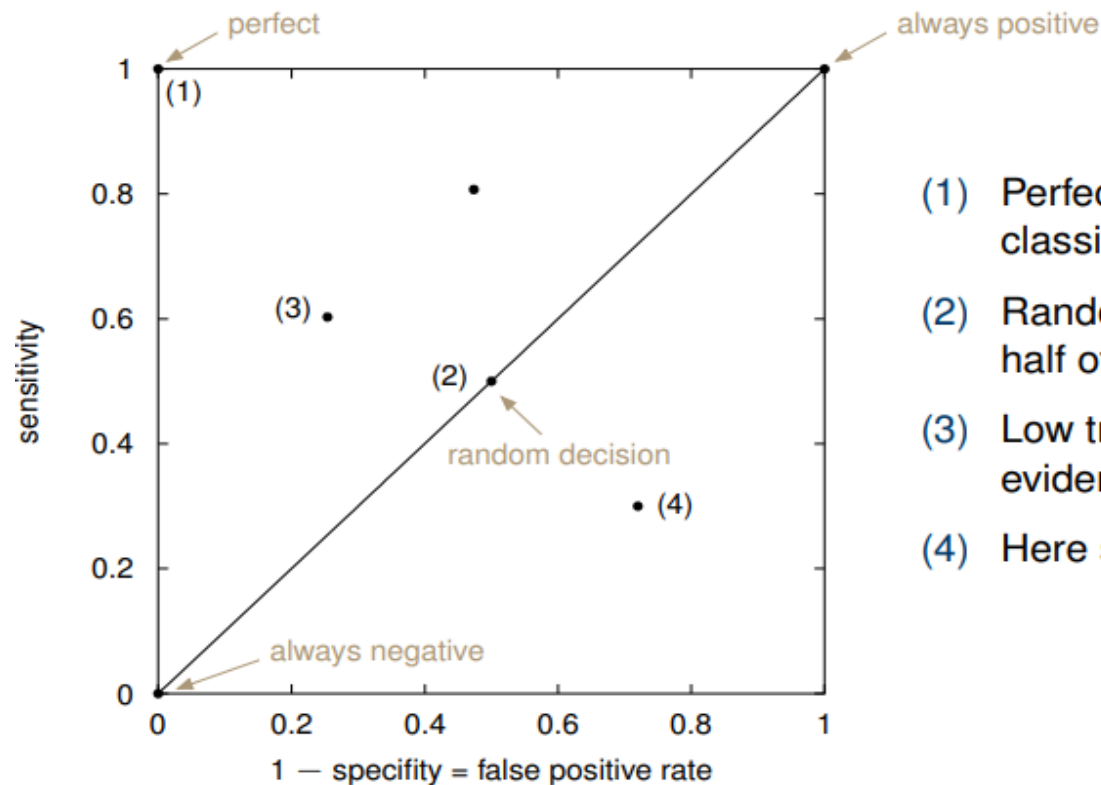
	Predict Correct	Score
1	Yes	0.95
2	Yes	0.86
3	Yes	0.69
4	No	0.65
5	Yes	0.59
6	No	0.52
7	No	0.39
8	No	0.28
9	Yes	0.15
10	No	0.06



Receiver Operating Characteristic (ROC): Since **TPR is equivalent to Sensitivity** and **FPR is equal to $1 - \text{specificity}$** , the ROC graph is sometimes called the sensitivity vs ($1 - \text{specificity}$) plot.



Receiver Operating Characteristic (ROC): Since **TPR is equivalent to Sensitivity** and **FPR is equal to $1 - \text{specificity}$** , the ROC graph is sometimes called the sensitivity vs ($1 - \text{specificity}$) plot.



- (1) Perfect classifier: no false positives, all negatives are classified as negatives
- (2) Random decision: half of positives are classified correctly, half of negatives are classified correctly
- (3) Low true positive rate, but lower false positive rate; strong evidence for positive classification
- (4) Here something goes really wrong!

Cross Validation in Machine Learning

Cross-validation is a statistical technique used to assess the performance and generalizability of a predictive model. It involves partitioning the data into subsets, training the model on some subsets, and validating it on the remaining subsets. When we have **limited data**, cross-validation becomes particularly valuable because it allows us to make the most efficient use of the available data to both train and validate the model.

Advantages:

- ❖ **Reduces Overfitting:** By training and validating the model on different subsets of data, cross-validation helps ensure that the model generalizes well to unseen data.
- ❖ **Provides Robust Performance Estimates:** It offers a more accurate estimate of model performance compared to a single train-test split.

Consider a scenario where you have 100 data points:

- ❖ **Without Cross-Validation:** If you split the data into 80% training (80 points) and 20% test (20 points), you might not have enough data to train a complex model or to get a reliable estimate of its performance.
- ❖ **With 5-Fold Cross-Validation:** You split the data into **5 folds** of 20 points each. Each fold serves as a test set once, and the model is trained on the remaining 80 points. This way, each data point is used for both training and testing, providing a more comprehensive evaluation.

1. k-Fold Cross-Validation:

- The dataset is divided into k equal-sized folds.
- The model is trained k times, each time using a different fold for validation and the remaining $k - 1$ folds for training.

2. Stratified k-Fold Cross-Validation:

- Similar to k-fold, but ensures that each fold is representative of the overall class distribution. This is particularly useful for imbalanced datasets.

3. Leave-One-Out Cross-Validation (LOOCV):

- A special case of k -fold where k equals the number of data points.
- Each fold consists of a single data point for validation, and the model is trained on the remaining data points.
- Provides an unbiased estimate but can be computationally expensive for large datasets.

4. Leave-P-Out Cross-Validation:

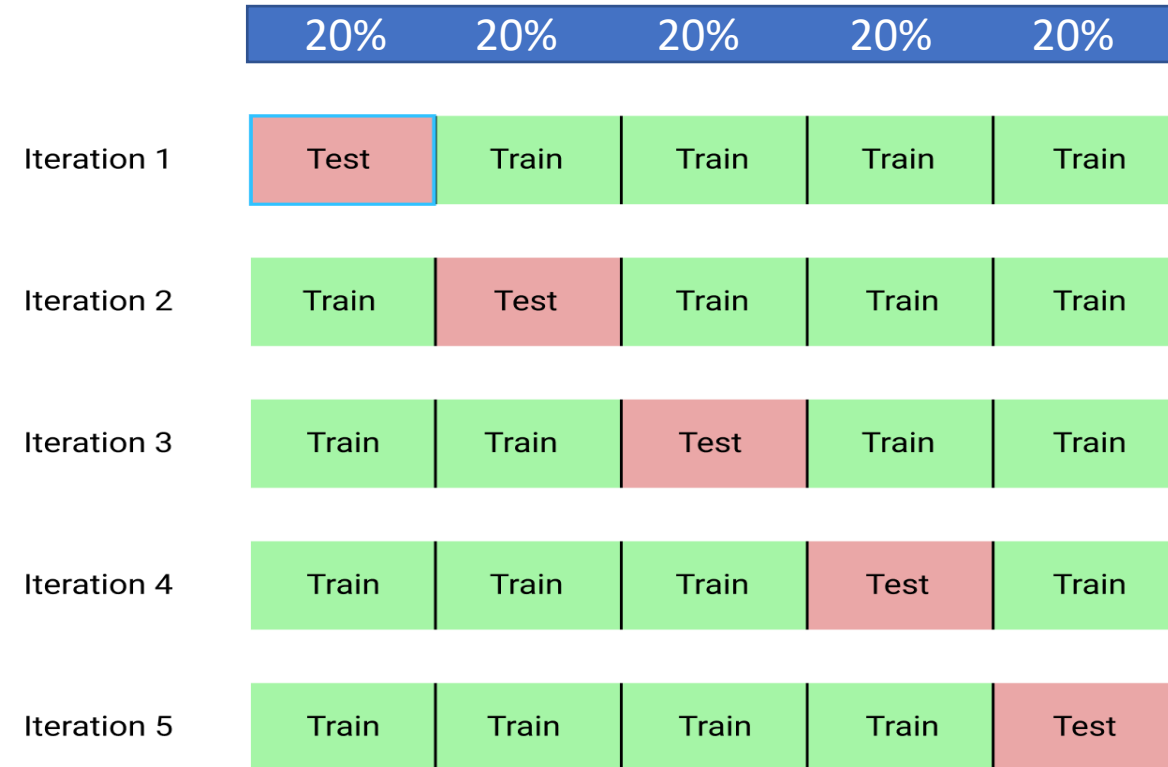
- Generalizes LOOCV by leaving p data points out for validation.
- Used less frequently due to computational complexity.

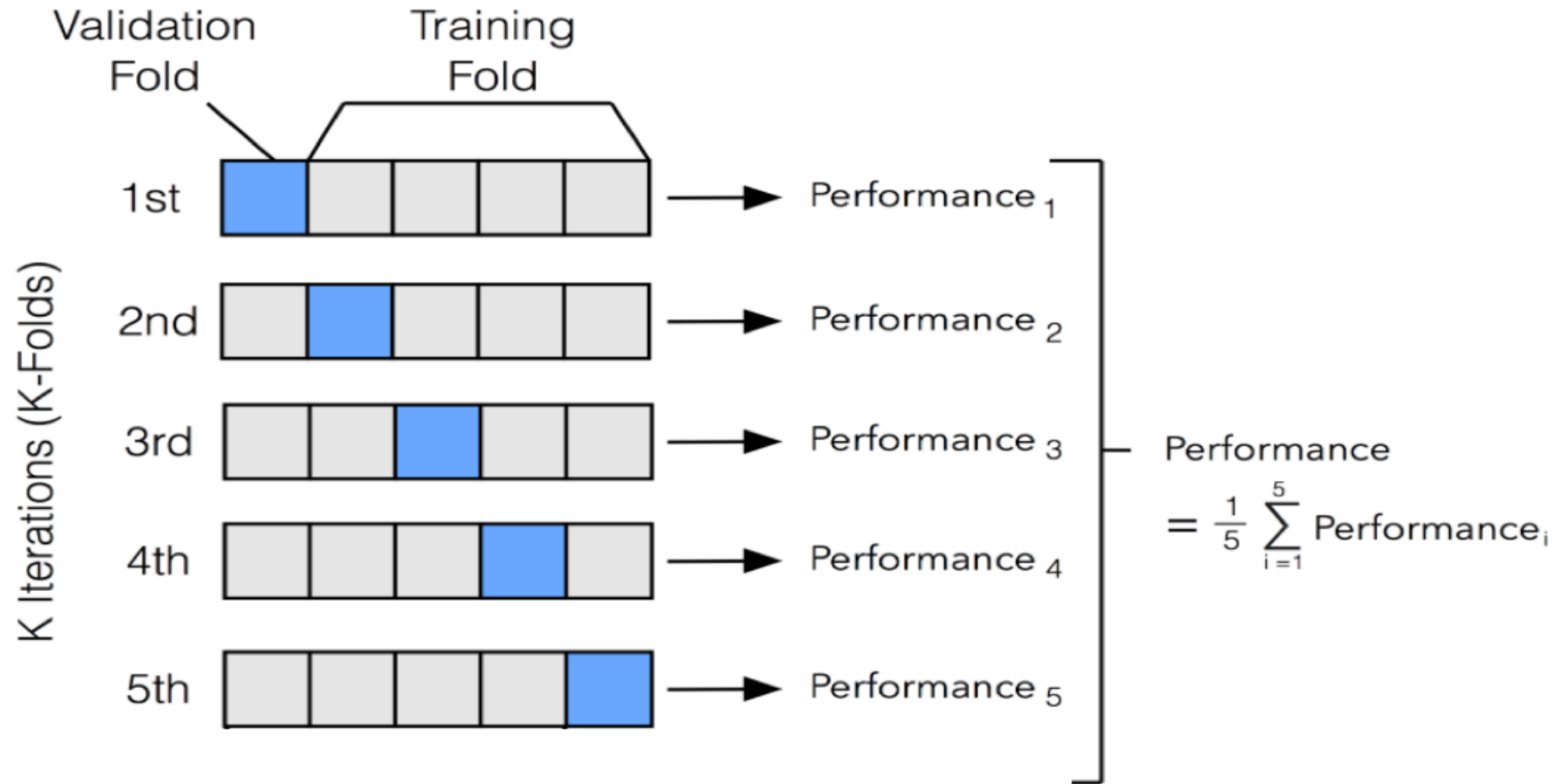
5. Time Series Cross-Validation:

- Used for time series data where the order of data points matters.
- The training set consists of observations from the past, and the validation set consists of observations from the future, ensuring no data leakage.

The general procedure is as follows:

1. Shuffle the dataset randomly.
2. Split the dataset into k groups
3. For each unique group:
 1. Take the group as a test data set
 2. Take the remaining groups as a training data set
 3. Fit a model on the training set and evaluate it on the test set
 4. Retain the evaluation score and discard the model
4. Summarize the results of the model using the sample of model evaluation scores

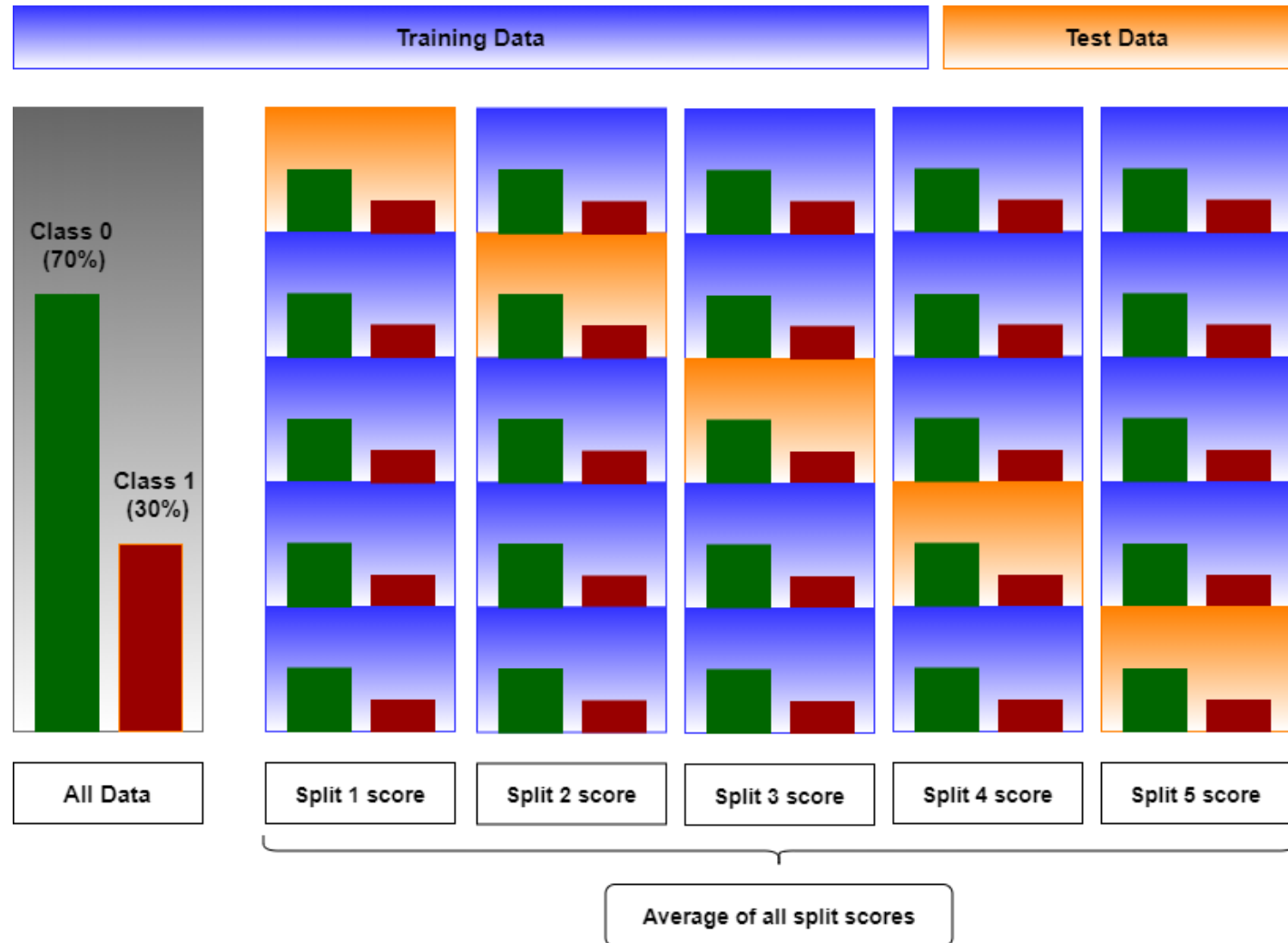




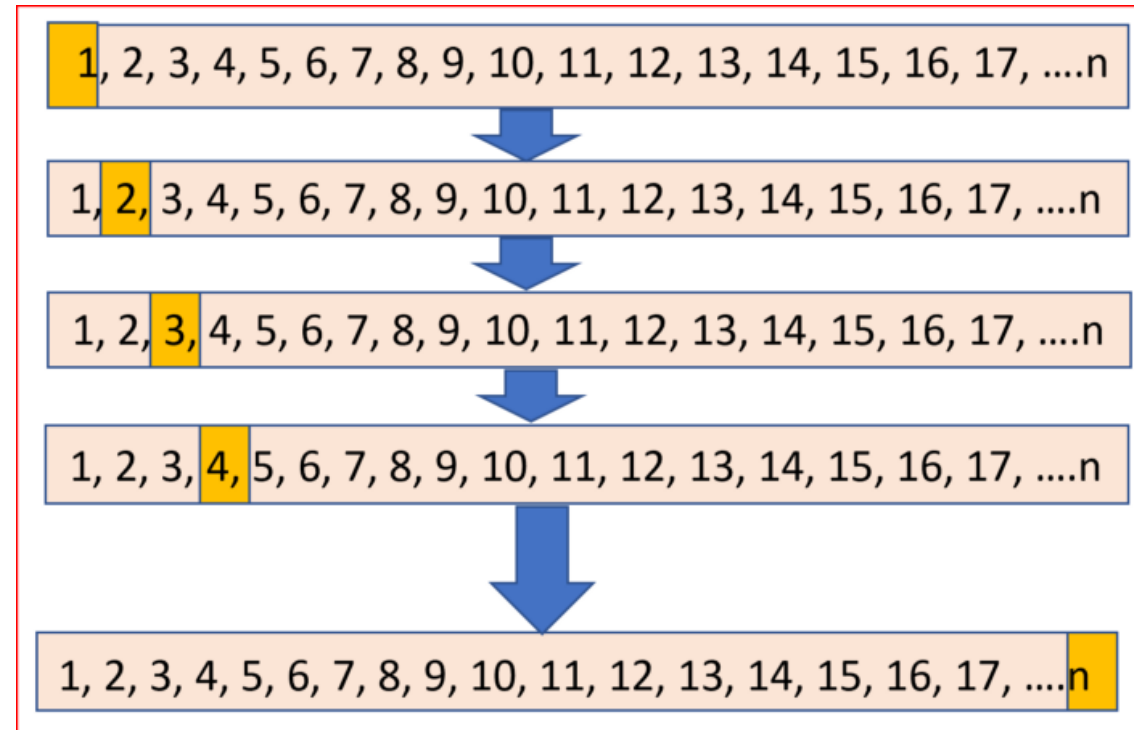
Stratified k-fold cross-validation is an extension of k-fold cross-validation, specifically designed to handle classification tasks where the class distribution needs to be preserved in each fold. This is particularly important for imbalanced datasets where some classes are underrepresented.

Step 01: Split the dataset into K equal-sized folds such that each fold has approximately the **same percentage of samples for each class** as the original dataset.

Step 02: Now Perform K-Fold Cross Validation in the same scenario.



Leave-One-Out Cross-Validation (LOOCV) is a case of k -fold cross-validation where the number of folds k equals the number of data points in the dataset. This means that each data point is used once as a single test instance while the remaining $n-1$ instances form the training set.



Let's Implement it with Python!